
User Manual

for MPC560XB DIO Driver

Document Number: UM13DIOASR4.0 Rev0003R1.0.1
Rev 1.2





Contents

Section number	Title	Page
Chapter 1		
Revision History		
Chapter 2		
Introduction		
2.1	Supported Derivatives.....	9
2.2	Overview.....	10
2.3	About this Manual.....	10
2.4	Acronyms and Definitions.....	11
2.5	Reference List.....	11
Chapter 3		
Driver		
3.1	Requirements.....	13
3.2	Driver Design Summary.....	13
3.3	Deviation from Requirements.....	13
3.4	Runtime Errors.....	14
3.5	Debug Variables.....	14
3.6	Software specification.....	14
3.6.1	Define Reference.....	15
3.6.1.1	Define DIO_E_PARAM_CONFIG.....	15
3.6.1.2	Define DIO_E_PARAM_INVALID_CHANNEL_ID.....	15
3.6.1.3	Define DIO_E_PARAM_INVALID_GROUP_ID.....	15
3.6.1.4	Define DIO_E_PARAM_INVALID_PORT_ID.....	16
3.6.1.5	Define DIO_E_PARAM_POINTER.....	16
3.6.1.6	Define DIO_FLIPCHANNEL_ID.....	16
3.6.1.7	Define DIO_GETVERSIONINFO_ID.....	17
3.6.1.8	Define DIO_INIT_ID.....	17
3.6.1.9	Define DIO_INSTANCE_ID.....	18
3.6.1.10	Define DIO_MASKEDWRITEPORT_ID.....	18

Section number	Title	Page
3.6.1.11	Define DIO_READCHANNEL_ID.....	18
3.6.1.12	Define DIO_READCHANNELGROUP_ID.....	19
3.6.1.13	Define DIO_READPORT_ID.....	19
3.6.1.14	Define DIO_WRITECHANNEL_ID.....	19
3.6.1.15	Define DIO_WRITECHANNELGROUP_ID.....	20
3.6.1.16	Define DIO_WRITEPORT_ID.....	20
3.6.2	Macros Reference.....	21
3.6.2.1	Macro Dio_LLD_ReadChannel.....	21
3.6.2.2	Macro Dio_LLD_ReadChannelGroup.....	21
3.6.2.3	Macro Dio_LLD_ReadPort.....	21
3.6.2.4	Macro Dio_LLD_WriteChannel.....	21
3.6.2.5	Macro Dio_LLD_WriteChannelGroup.....	22
3.6.2.6	Macro Dio_LLD_WritePort.....	22
3.6.2.7	Macro SIUL_GPDI.....	22
3.6.2.8	Macro SIUL_GPDO.....	22
3.6.2.9	Macro SIUL_IFMC.....	22
3.6.2.10	Macro SIUL_MPGPDO.....	22
3.6.2.11	Macro SIUL_PCR.....	22
3.6.2.12	Macro SIUL_PGPDI.....	23
3.6.2.13	Macro SIUL_PGPDO.....	23
3.6.2.14	Macro SIUL_PSMI.....	23
3.6.3	Enum Reference.....	23
3.6.4	Function Reference.....	23
3.6.4.1	Function Dio_FlipChannel.....	23
3.6.4.2	Function Dio_GetVersionInfo.....	24
3.6.4.3	Function Dio_Init.....	24
3.6.4.4	Function Dio_MaskedWritePort.....	25
3.6.4.5	Function Dio_ReadChannel.....	25
3.6.4.6	Function Dio_ReadChannelGroup.....	26

Section number	Title	Page
3.6.4.7	Function Dio_ReadPort.....	27
3.6.4.8	Function Dio_WriteChannel.....	27
3.6.4.9	Function Dio_WriteChannelGroup.....	28
3.6.4.10	Function Dio_WritePort.....	28
3.6.5	Structs Reference.....	29
3.6.6	Types Reference.....	29
3.6.7	Variables Reference.....	29
3.6.7.1	Variable Dio_CurrentConfigPtr.....	29

Chapter 4 Tresos Configuration Plug-in

4.1	Configuration elements of Dio.....	31
4.2	Form IMPLEMENTATION_CONFIG_VARIANT.....	31
4.3	Form DioGeneral.....	32
4.3.1	DioDevErrorDetect (DioGeneral).....	32
4.3.2	DioFlipChannelApi (DioGeneral).....	33
4.3.3	DioVersionInfoApi (DioGeneral).....	33
4.3.4	DioReversePortBits (DioGeneral).....	33
4.3.5	DioMaskedWritePortApi (DioGeneral).....	34
4.3.6	DioReadZeroForUndefinedPortPins (DioGeneral).....	34
4.4	Form CommonPublishedInformation.....	35
4.4.1	ArReleaseMajorVersion (CommonPublishedInformation).....	35
4.4.2	ArReleaseMinorVersion (CommonPublishedInformation).....	36
4.4.3	ArReleaseRevisionVersion (CommonPublishedInformation).....	36
4.4.4	ModuleId (CommonPublishedInformation).....	37
4.4.5	SwMajorVersion (CommonPublishedInformation).....	37
4.4.6	SwMinorVersion (CommonPublishedInformation).....	38
4.4.7	SwPatchVersion (CommonPublishedInformation).....	38
4.4.8	VendorApiInfix (CommonPublishedInformation).....	39
4.4.9	VendorId (CommonPublishedInformation).....	39

Section number	Title	Page
4.5	Form DioConfig.....	40
4.5.1	Form DioPort.....	40
4.5.1.1	DioPortId (DioPort).....	41
4.5.1.2	Form DioChannel.....	41
4.5.1.2.1	DioChannelId (DioChannel).....	42
4.5.1.3	Form DioChannelGroup.....	42
4.5.1.3.1	DioChannelGroupIdentification (DioChannelGroup).....	43
4.5.1.3.2	DioPortMask (DioChannelGroup).....	43
4.5.1.3.3	DioPortOffset (DioChannelGroup).....	43

Chapter 1

Revision History

Table 1-1. Revision History

Revision	Date	Author	Description
1.0	27-Jan-2012	Dario Arcaro	Initial Draft for Beta 0.9.0
1.1	17-May-2013	Sunil Khatri	Bolero Small RTM 1.0.0
1.2	02-June-2014	François Penalosa	Bolero Small RTM 1.0.1



Chapter 2

Introduction

This User Manual describes Freescale Semiconductor AUTOSAR Digital Input Output (Dio) for MPC5607B .

AUTOSAR Dio driver configuration parameters and deviations from the specification are described in Dio Driver chapter of this document. AUTOSAR Dio driver requirements and APIs are described in the AUTOSAR Dio driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of Freescale Semiconductor .

Table 2-1. MPC5607B Derivatives

Freescale Semiconductor	mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100 mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkll_lqfp100 mpc5606bxlq_lqfp144
-------------------------	---

Table 2-1. MPC5607B Derivatives

	mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208
--	---

All of the above microcontroller devices are collectively named as MPC5607B .

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About this Manual

This Technical Reference employs the following typographical conventions:

Boldface type: Bold is used for important terms, notes and warnings.

Italic font: Italic typeface is used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

2.4 Acronyms and Definitions

Table 2-2. Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
DEM	Diagnostic Event Manager
DET	Development Error Tracer
C/CPP	C and C++ Source Code
VLE	Variable Length Encoding
N/A	Not Applicable
MCU	Micro Controller Unit

2.5 Reference List

Table 2-3. Reference List

#	Title	Version
1	AUTOSAR 4.0 Rev0003Dio Driver Software Specification Document.	2.5.0
2	MPC5607B Reference Manual	Rev. 7.2, May 2012
3	MPC5604B Reference Manual	Rev. 8.2, September 2013
4	5606B Reference Manual	Rev. 2, May 2014
5	5602D Reference Manual	Rev. 4.2, Dec 2013
6	Bolero 1.5M cut 2.2 Errata Sheet Reference Manual	MPC5607B_1M03Y - Rev. 12Dec2013
7	Bolero 512k cut 2.3 Errata Sheet Reference Manual	MPC5604B_2M27V - Rev. 26 FEB 2014
8	Bolero 256k cut 1.1 Errata Sheet Reference Manual	MPC5602D_1M18Y - Rev. , 12 DEC 2013

Chapter 3 Driver

3.1 Requirements

Requirements for this driver are detailed in the AUTOSAR 4.0 Rev0003Dio Driver Software Specification document (See Table [Reference List](#)).

3.2 Driver Design Summary

The DIO Driver provides services for reading and writing to/from:

- DIO Channels (Pins)
- DIO Ports
- DIO Channel Groups

The behaviour of those services is synchronous. This module works on pins and ports which are configured by the PORT driver for this purpose. For this reason, there is no configuration and initialization of this port structure in the DIO Driver.

3.3 Deviation from Requirements

The driver deviates from the AUTOSAR Dio Driver software specification in some places. Table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, or out of scope for the Dio driver. Table [Table 3-1](#) provides Status column description.

Table 3-1. Deviations Status Column Description

Term	Definition
N/A	Not available
N/T	Not testable
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the AUTOSAR requirements that are not fully implemented, implemented differently, or out of scope for the driver.

Table 3-2. Driver Deviations Table

Requirement	Status	Description	Notes
DIO067	N/I	The Dio module shall report production errors to the Diagnostic Event Manager.	Production errors are not available in Dio
DIO104	Note	When reading a port which is smaller than the Dio_PortType using the Dio_ReadPort function (see [DIO103]), the function shall set the bits corresponding to undefined port pins to 0.	The implementation of this requirement can be disabled using the configuration parameter DioReadZeroForUndefinedPortPins in order to improve performance.

3.4 Runtime Errors

This driver doesn't generate any runtime error.

3.5 Debug Variables

The following global variables are available at runtime for debugging and inspection according to general requirement BSW00442.

Table 3-3. Debug Variables

Variable	Type	Description
Dio_CurrentConfigPtr	Dio_ConfigType *	Pointer to the current DIO driver configuration structure.

3.6 Software specification

The following sections contains driver software specifications.

3.6.1 Define Reference

Constants supported by the Dio driver.

3.6.1.1 Define DIO_E_PARAM_CONFIG

API service called with "NULL pointer" parameter.

Implements: DDIO03013, DDIO03015.

Table 3-4. Define DIO_E_PARAM_CONFIG Description

Name	DIO_E_PARAM_CONFIG
Initializer	((uint8)0x10)

3.6.1.2 Define DIO_E_PARAM_INVALID_CHANNEL_ID

Invalid channel name requested.

Implements: DDIO03013, DDIO03014.

Table 3-5. Define DIO_E_PARAM_INVALID_CHANNEL_ID Description

Name	DIO_E_PARAM_INVALID_CHANNEL_ID
Initializer	((uint8)0x0A)

3.6.1.3 Define DIO_E_PARAM_INVALID_GROUP_ID

Invalid ChannelGroup id passed.

Implements: DDIO03013, DDIO03017.

Table 3-6. Define DIO_E_PARAM_INVALID_GROUP_ID Description

Name	DIO_E_PARAM_INVALID_GROUP_ID
Initializer	((uint8)0x1F)

3.6.1.4 Define DIO_E_PARAM_INVALID_PORT_ID

Invalid port name requested.

Implements: DDIO03013, DDIO03016.

Table 3-7. Define DIO_E_PARAM_INVALID_PORT_ID Description

Name	DIO_E_PARAM_INVALID_PORT_ID
Initializer	((uint8)0x14)

3.6.1.5 Define DIO_E_PARAM_POINTER

API service called with a NULL pointer.

Details:

In case of this error, the API service shall return immediately without any further action, beside reporting this development error.

Implements: DDIO03013, DDIO03018.

Table 3-8. Define DIO_E_PARAM_POINTER Description

Name	DIO_E_PARAM_POINTER
Initializer	((uint8)0x20)

3.6.1.6 Define DIO_FLIPCHANNEL_ID

API service ID for `Dio_Init()` function.

Details:

Parameters used when raising an error/exception.

Implements: DDIO03027.

Table 3-9. Define DIO_FLIPCHANNEL_ID Description

Name	DIO_FLIPCHANNEL_ID
Initializer	((uint8)0x11)

3.6.1.7 Define DIO_GETVERSIONINFO_ID

API service ID for DIO Get Version() Info function.

Details:

Parameters used when raising an error/exception.

Implements: DDIO03025.

Table 3-10. Define DIO_GETVERSIONINFO_ID Description

Name	DIO_GETVERSIONINFO_ID
Initializer	((uint8)0x12)

3.6.1.8 Define DIO_INIT_ID

API service ID for `Dio_Init()` function.

Details:

Parameters used when raising an error/exception.

Implements: DDIO03026.

Table 3-11. Define DIO_INIT_ID Description

Name	DIO_INIT_ID
Initializer	((uint8)0x10)

3.6.1.9 Define DIO_INSTANCE_ID

Instance ID of the Dio driver.

Implements:

Table 3-12. Define DIO_INSTANCE_ID Description

Name	DIO_INSTANCE_ID
Initializer	((uint8)0x00)

3.6.1.10 Define DIO_MASKEDWRITEPORT_ID

API service ID for `Dio_MaskedWritePort()` function.

Details:

Parameters used when raising an error/exception.

Implements: DDIO03028.

Table 3-13. Define DIO_MASKEDWRITEPORT_ID Description

Name	DIO_MASKEDWRITEPORT_ID
Initializer	((uint8)0x06)

3.6.1.11 Define DIO_READCHANNEL_ID

API service ID for `Dio_ReadChannel()` function.

Details:

Parameters used when raising an error/exception.

Implements: DDIO03019.

Table 3-14. Define DIO_READCHANNEL_ID Description

Name	DIO_READCHANNEL_ID
Initializer	((uint8)0x00)

3.6.1.12 Define DIO_READCHANNELGROUP_ID

API service ID for `Dio_ReadChannel()` Group function.

Details:

Parameters used when raising an error/exception.

Implements: DDIO03023.

Table 3-15. Define DIO_READCHANNELGROUP_ID Description

Name	DIO_READCHANNELGROUP_ID
Initializer	((uint8)0x04)

3.6.1.13 Define DIO_READPORT_ID

API service ID for `Dio_ReadPort()` function.

Details:

Parameters used when raising an error/exception.

Implements: DDIO03021.

Table 3-16. Define DIO_READPORT_ID Description

Name	DIO_READPORT_ID
Initializer	((uint8)0x02)

3.6.1.14 Define DIO_WRITECHANNEL_ID

API service ID for `Dio_WriteChannel()` function.

Details:

Parameters used when raising an error/exception.

Implements: DDIO03020.

Table 3-17. Define DIO_WRITECHANNEL_ID Description

Name	DIO_WRITECHANNEL_ID
Initializer	((uint8)0x01)

3.6.1.15 Define DIO_WRITECHANNELGROUP_ID

API service ID for `Dio_WriteChannel()` Group function.

Details:

Parameters used when raising an error/exception.

Implements: DDIO03024.

Table 3-18. Define DIO_WRITECHANNELGROUP_ID Description

Name	DIO_WRITECHANNELGROUP_ID
Initializer	((uint8)0x05)

3.6.1.16 Define DIO_WRITEPORT_ID

API service ID for `Dio_WritePort()` function.

Details:

Parameters used when raising an error/exception.

Implements: DDIO03022.

Table 3-19. Define DIO_WRITEPORT_ID Description

Name	DIO_WRITEPORT_ID
Initializer	((uint8)0x03)

3.6.2 Macros Reference

Macros supported by the Dio driver.

3.6.2.1 Macro Dio_LLD_ReadChannel

Mapping macro between low level layer and high level layer.

Implements: DDIO04002.

Prototype: Dio_LLD_ReadChannel(ChannelId)

3.6.2.2 Macro Dio_LLD_ReadChannelGroup

Mapping macro between low level layer and high level layer.

Implements: DDIO04003.

Prototype: Dio_LLD_ReadChannelGroup(ChannelGroupIdPtr)

3.6.2.3 Macro Dio_LLD_ReadPort

Mapping macro between low level layer and high level layer.

Implements: DDIO04004.

Prototype: Dio_LLD_ReadPort(PortId)

3.6.2.4 Macro Dio_LLD_WriteChannel

Mapping macro between low level layer and high level layer.

Implements: DDIO04005.

Prototype: `Dio_LLD_WriteChannel(ChannelId,ChannelLevel)`

3.6.2.5 Macro Dio_LLD_WriteChannelGroup

Mapping macro between low level layer and high level layer.

Implements: DDIO04006.

Prototype: `Dio_LLD_WriteChannelGroup(ChannelGroupIdPtr,Level)`

3.6.2.6 Macro Dio_LLD_WritePort

Mapping macro between low level layer and high level layer.

Implements: DDIO04007.

Prototype: `Dio_LLD_WritePort(PortId,PortLevel)`

3.6.2.7 Macro SIUL_GPDI

Prototype: `SIUL_GPDI(PIN)`

3.6.2.8 Macro SIUL_GPDO

Prototype: `SIUL_GPDO(PIN)`

3.6.2.9 Macro SIUL_IFMC

Prototype: `SIUL_IFMC(IFMC_OFFSET)`

3.6.2.10 Macro SIUL_MPGPDO

Prototype: `SIUL_MPGPDO(MPGPDO_OFFSET)`

3.6.2.11 Macro SIUL_PCR

Prototype: `SIUL_PCR(PIN)`

3.6.2.12 Macro SIUL_PGPDI

Prototype: `SIUL_PGPDI(PGPDI_OFFSET)`

3.6.2.13 Macro SIUL_PGPDO

Prototype: `SIUL_PGPDO(PGPDO_OFFSET)`

3.6.2.14 Macro SIUL_PSMI

Prototype: `SIUL_PSMI(PSMI_OFFSET)`

3.6.3 Enum Reference

Enumeration of all constants supported by the Dio driver.

3.6.4 Function Reference

Functions supported by the Dio driver.

3.6.4.1 Function Dio_FlipChannel

Inverts the level of a channel.

Details:

If the specified channel is configured as an output channel, this function shall invert the level of the specified channel. If the specified channel is configured as an input channel, this function shall have no influence on the physical output and on the result of the next read service.

Return: Returns the level of the corresponding pin as `STD_HIGH` or `STD_LOW`.

Pre: This function can be used only if DIO_FLIP_CHANNEL_API has been enabled.

Implements: DDIO03001.

Prototype: `Dio_LevelType Dio_FlipChannel(const Dio_ChannelType ChannelId);`

Table 3-20. Dio_FlipChannel Arguments

Type	Name	Direction	Description
const Dio_ChannelType	ChannelId	input	Specifies the required channel id.

Table 3-21. Dio_FlipChannel Return Values

Name	Description
STD_HIGH	The logical level of the corresponding pin is 1.
STD_LOW	The logical level of the corresponding Pin is 0.

3.6.4.2 Function Dio_GetVersionInfo

Service to get the version information of this module.

Details:

The `Dio_GetVersionInfo()` function shall return the version information of this module. The version information includes:

- Module Id.
- Vendor Id.
- Vendor specific version numbers.

Pre: This function can be used only if DIO_VERSION_INFO_API has been enabled.

Implements: DDIO03030.

Prototype: `void Dio_GetVersionInfo(Std_VersionInfoType *versioninfo);`

Table 3-22. Dio_GetVersionInfo Arguments

Type	Name	Direction	Description
Std_VersionInfoType *	versioninfo	input, output	Pointer to where to store the version information of this module.

3.6.4.3 Function Dio_Init

Driver initialization.

Details:

This function must be invoked before the driver can be used.

Implements: DDIO03012.

Prototype: void Dio_Init(const Dio_ConfigType *ConfigPtr);

Table 3-23. Dio_Init Arguments

Type	Name	Direction	Description
const Dio_ConfigType *	ConfigPtr	input	Pointer to the configuration data.

3.6.4.4 Function Dio_MaskedWritePort

DIO Mask write port using mask.

Details:

Writes a DIO port with masked value.

Pre: This function can be used only if DIO_MASKEDWRITEPORT_API has been enabled.

Implements: DDIO03031.

Prototype: void Dio_MaskedWritePort(const Dio_PortType PortId, const Dio_PortLevelType Level, const Dio_PortLevelType Mask);

Table 3-24. Dio_MaskedWritePort Arguments

Type	Name	Direction	Description
const Dio_PortType	PortId	input	Specifies the required port id.
const Dio_PortLevelType	Level	input	Specifies the required levels for the port pins.
const Dio_PortLevelType	Mask	input	Specifies the Mask value of the port.

3.6.4.5 Function Dio_ReadChannel

Returns the value of the specified DIO channel.

Details:

This function returns the value of the specified DIO channel.

Return: Returns the level of the corresponding pin as STD_HIGH or STD_LOW.

Implements: DDIO03002.

Prototype: `Dio_LevelType Dio_ReadChannel(const Dio_ChannelType ChannelId);`

Table 3-25. Dio_ReadChannel Arguments

Type	Name	Direction	Description
const Dio_ChannelType	ChannelId	input	Specifies the required channel id.

Table 3-26. Dio_ReadChannel Return Values

Name	Description
STD_HIGH	The logical level of the corresponding pin is 1.
STD_LOW	The logical level of the corresponding Pin is 0.

3.6.4.6 Function Dio_ReadChannelGroup

This service reads a subset of the adjoining bits of a port.

Details:

This function will read a subset of adjoining bits of a port (channel group).

Return: The channel group levels.

Implements: DDIO03003.

Prototype: `Dio_PortLevelType Dio_ReadChannelGroup(const Dio_ChannelGroupType *ChannelGroupIdPtr);`

Table 3-27. Dio_ReadChannelGroup Arguments

Type	Name	Direction	Description
const Dio_ChannelGroupType *	ChannelGroupIdPtr	input	Pointer to the channel group.

3.6.4.7 Function Dio_ReadPort

Returns the level of all channels of specified port.

Details:

This function will return the level of all channels belonging to the specified port.

Return: Levels of all channels of specified port.

Implements: DDIO03004.

Prototype: `Dio_PortLevelType Dio_ReadPort(const Dio_PortType PortId);`

Table 3-28. Dio_ReadPort Arguments

Type	Name	Direction	Description
const Dio_PortType	PortId	input	Specifies the required port id.

3.6.4.8 Function Dio_WriteChannel

Sets the level of a channel.

Details:

If the specified channel is configured as an output channel, this function shall set the specified level on the specified channel. If the specified channel is configured as an input channel, this function shall have no influence on the physical output and on the result of the next read service.

Implements: DDIO03005.

Prototype: `void Dio_WriteChannel(const Dio_ChannelType ChannelId, const Dio_LevelType Level);`

Table 3-29. Dio_WriteChannel Arguments

Type	Name	Direction	Description
const Dio_ChannelType	ChannelId	input	Specifies the required channel id.
const Dio_LevelType	Level	input	Specifies the channel desired level.

3.6.4.9 Function Dio_WriteChannelGroup

Sets a subset of the adjoining bits of a port to the specified levels.

Details:

This function will set a subset of adjoining bits of a port (channel group) to the specified levels without changing the remaining channels of the port and channels that are configured as input. This function will do the masking of the channels and will do the shifting so that the values written by the function are aligned to the LSB.

Implements: DDIO03006.

Prototype: void Dio_WriteChannelGroup(const Dio_ChannelGroupType *ChannelGroupIdPtr, const Dio_PortLevelType Level);

Table 3-30. Dio_WriteChannelGroup Arguments

Type	Name	Direction	Description
const Dio_ChannelGroupType *	ChannelGroupIdPtr	input	Pointer to the channel group.
const Dio_PortLevelType	Level	input	Desired levels for the channel group.

3.6.4.10 Function Dio_WritePort

Sets the value of a port.

Details:

This function will set the specified value on the specified port.

Implements: DDIO03007.

Prototype: void Dio_WritePort(const Dio_PortType PortId, const Dio_PortLevelType Level);

Table 3-31. Dio_WritePort Arguments

Type	Name	Direction	Description
const Dio_PortType	PortId	input	Specifies the required port id.
const Dio_PortLevelType	Level	input	Specifies the required levels for the port pins.

3.6.5 Structs Reference

Data structures supported by the Dio driver.

3.6.6 Types Reference

Types structures supported by the Dio driver.

3.6.7 Variables Reference

Variables supported by the Dio driver.

3.6.7.1 Variable Dio_CurrentConfigPtr

Current configuration pointer.

Implements: DDIO03029.

Declaration:

```
const Dio_ConfigType* Dio_CurrentConfigPtr
```


Chapter 4

Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the Dio Driver. The most of the parameters are described below.

4.1 Configuration elements of Dio

Included forms :

- IMPLEMENTATION_CONFIG_VARIANT
- DioGeneral
- CommonPublishedInformation
- DioConfig

Table 4-1. Revision table

Revision	Date
4.1.0	2010-12-03

4.2 Form IMPLEMENTATION_CONFIG_VARIANT

VariantPreCompile: Only precompile time configuration parameters.

VariantPostBuild: Mix of precompile and postbuild time configuration parameters.

If Config Variant = VariantPreCompile, the files Dio_Cfg.h and Dio_Cfg.c shall be used.

If Config Variant = VariantPostBuild, the files Dio_Cfg.h and Dio_PBcfg.c shall be used.

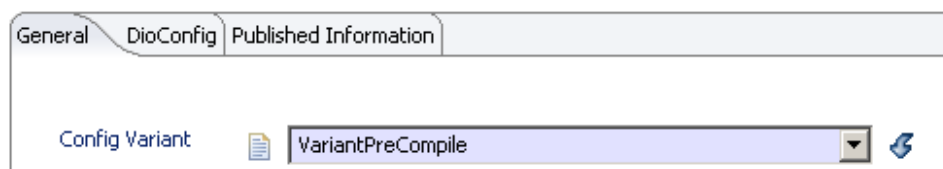


Figure 4-1. Tresos Plugin snapshot for IMPLEMENTATION_CONFIG_VARIANT form.

Table 4-2. Attribute IMPLEMENTATION_CONFIG_VARIANT detailed description

Property	Value
Label	Config Variant
Type	ENUMERATION
Default	VariantPostBuild
Range	VariantPostBuild VariantPreCompile

4.3 Form DioGeneral

General DIO module configuration parameters.

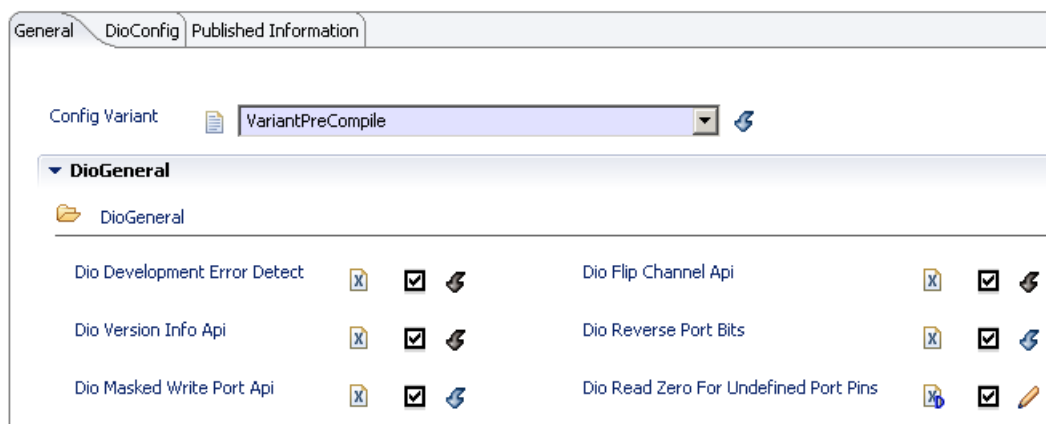


Figure 4-2. Tresos Plugin snapshot for DioGeneral form.

4.3.1 DioDevErrorDetect (DioGeneral)

Switches the Development Error Detection and Notification ON or OFF

Table 4-3. Attribute DioDevErrorDetect (DioGeneral) detailed description

Property	Value
Label	Dio Development Error Detect
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	false

4.3.2 DioFlipChannelApi (DioGeneral)

Adds / removes the service Dio_FlipChannel() from the code.

Table 4-4. Attribute DioFlipChannelApi (DioGeneral) detailed description

Property	Value
Label	Dio Flip Channel Api
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	true

4.3.3 DioVersionInfoApi (DioGeneral)

Adds / removes the service Dio_GetVersionInfo() from the code.

Table 4-5. Attribute DioVersionInfoApi (DioGeneral) detailed description

Property	Value
Label	Dio Version Info Api
Type	BOOLEAN
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	true

4.3.4 DioReversePortBits (DioGeneral)

If this box is checked the bits written to defined ports will be reversed, writing 3 to PORTA with checkbox disabled will set pins 14 and 15, writing 3 to PORTA with checkbox enabled will set pins 0 and 1.

Table 4-6. Attribute DioReversePortBits (DioGeneral) detailed description

Property	Value
Label	Dio Reverse Port Bits
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.3.5 DioMaskedWritePortApi (DioGeneral)

Defines whether the driver function Dio_MaskedWritePort () will be included at compile time or excluded.

True - Dio_MaskedWritePort () API enabled.

False - Dio_MaskedWritePort () API disabled.

Table 4-7. Attribute DioMaskedWritePortApi (DioGeneral) detailed description

Property	Value
Label	Dio Masked Write Port Api
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	false

4.3.6 DioReadZeroForUndefinedPortPins (DioGeneral)

Defines whether the Dio_ReadPort() function includes the capability to read the undefined port pins as 0.

True - Enables the Dio_ReadPort() functionality to read the undefined port pins as 0.

False - Disables the Dio_ReadPort() functionality to read the undefined port pins as 0 (Supports the normal functionality with Dio_ReadPort()).

Table 4-8. Attribute DioReadZeroForUndefinedPortPins (DioGeneral) detailed description

Property	Value
Label	Dio Read Zero For Undefined Port Pins
Type	BOOLEAN
Origin	Custom
Symbolic Name	false
Default	true

4.4 Form CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

CommonPublishedInformation	
CommonPublishedInformation	
AUTOSAR Major Version	4
AUTOSAR Minor Version	0
AUTOSAR Release Revision Version	3
Module Id	120
Software Major Version	0
Software Minor Version	9
Software Patch Version	0
Vendor Api Infix	
Vendor Id	43

Figure 4-3. Tresos Plugin snapshot for CommonPublishedInformation form.

4.4.1 ArReleaseMajorVersion (CommonPublishedInformation)

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Table 4-9. Attribute ArReleaseMajorVersion (CommonPublishedInformation) detailed description

Property	Value
Label	AUTOSAR Major Version
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	4
Invalid	Range >=4 <=4

4.4.2 ArReleaseMinorVersion (CommonPublishedInformation)

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Table 4-10. Attribute ArReleaseMinorVersion (CommonPublishedInformation) detailed description

Property	Value
Label	AUTOSAR Minor Version
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	0
Invalid	Range >=0 <=0

4.4.3 ArReleaseRevisionVersion (CommonPublishedInformation)

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Table 4-11. Attribute ArReleaseRevisionVersion (CommonPublishedInformation) detailed description

Property	Value
Label	AUTOSAR Release Revision Version
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	3
Invalid	Range >=3 <=3

4.4.4 ModuleId (CommonPublishedInformation)

Module ID of this module from Module List.

Table 4-12. Attribute ModuleId (CommonPublishedInformation) detailed description

Property	Value
Label	Module Id
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	120
Invalid	Range >=120 <=120

4.4.5 SwMajorVersion (CommonPublishedInformation)

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Table 4-13. Attribute SwMajorVersion (CommonPublishedInformation) detailed description

Property	Value
Label	Software Major Version
Type	INTEGER_LABEL
Origin	Custom

Table continues on the next page...

Table 4-13. Attribute SwMajorVersion (CommonPublishedInformation) detailed description (continued)

Property	Value
Symbolic Name	false
Default	1
Invalid	Range >=1 <=1

4.4.6 SwMinorVersion (CommonPublishedInformation)

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Table 4-14. Attribute SwMinorVersion (CommonPublishedInformation) detailed description

Property	Value
Label	Software Minor Version
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	0
Invalid	Range >=0 <=0

4.4.7 SwPatchVersion (CommonPublishedInformation)

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Table 4-15. Attribute SwPatchVersion (CommonPublishedInformation) detailed description

Property	Value
Label	Software Patch Version
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	1

Table continues on the next page...

Table 4-15. Attribute SwPatchVersion (CommonPublishedInformation) detailed description (continued)

Property	Value
Invalid	Range >=1 <=1

4.4.8 VendorApiInfix (CommonPublishedInformation)

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name. This parameter is used to specify the vendor specific name. In total, the implementation specific name is generated as follows:

<ModuleName>_>VendorId>_<VendorApiInfix><Api name from SWS>. E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name Can_Write defined in the SWS will translate to Can_123_v11r456Write. This parameter is mandatory for all modules with upper multiplicity > 1. It shall not be used for modules with upper multiplicity =1.

Table 4-16. Attribute VendorApiInfix (CommonPublishedInformation) detailed description

Property	Value
Label	Vendor Api Infix
Type	STRING_LABEL
Origin	Custom
Symbolic Name	false
Default	
Enable	false

4.4.9 VendorId (CommonPublishedInformation)

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Table 4-17. Attribute VendorId (CommonPublishedInformation) detailed description

Property	Value
Label	Vendor Id

Table continues on the next page...

Table 4-17. Attribute VendorId (CommonPublishedInformation) detailed description (continued)

Property	Value
Type	INTEGER_LABEL
Origin	Custom
Symbolic Name	false
Default	43
Invalid	Range >=43 <=43

4.5 Form DioConfig

Included forms :

- [Form DioPort](#)

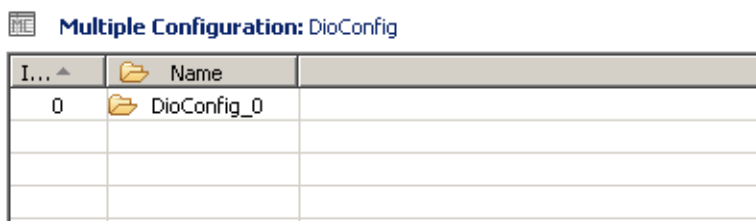


Figure 4-4. TRESOS Plugin snapshot for DioConfig form.

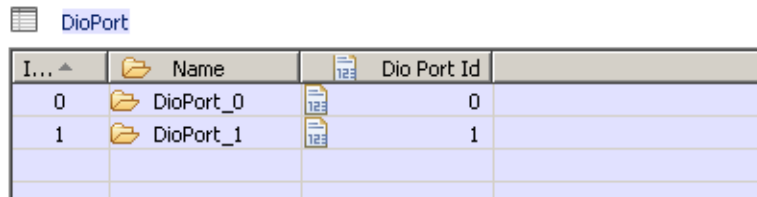
4.5.1 Form DioPort

Configuration of individual DIO ports, consisting of channels and possible channel groups. The single DIO channel levels inside a DIO port represent a bit in the DIO port value. A channel group is a formal logical combination of several adjoining DIO channels within a DIO port. The configuration process for Dio module shall provide symbolic names for each configured DIO channel, port and group.

Is included by form : [Form DioConfig](#)

Included forms :

- [Form DioChannel](#)
- [Form DioChannelGroup](#)



I...	Name	Dio Port Id
0	DioPort_0	0
1	DioPort_1	1

Figure 4-5. Tressos Plugin snapshot for DioPort form.

4.5.1.1 DioPortId (DioPort)

Numeric identifier of the DIO port. Symbolic names will be generated for each port pin id for the pins which being used for configuration. NOTE: Use the following values to configure different ports.

- PortA=0
- PortB=1
- PortC=2
- PortD=3
- PortE=4
- PortF=5
- PortG=6
- PortH=7
- PortI=8
- PortJ=9

Table 4-18. Attribute DioPortId (DioPort) detailed description

Property	Value
Label	Dio Port Id
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	true
Default	0
Invalid	Range <=9 >=0

4.5.1.2 Form DioChannel

Configuration of an individual DIO channel. Symbolic names will be generated for each channel. A general purpose digital IO pin represents a DIO channel which will be having value either STD_HIGH or STD_LOW.

Is included by form : [Form DioPort](#)

Figure 4-6. Tresos Plugin snapshot for DioChannel form.

4.5.1.2.1 DioChannelId (DioChannel)

Channel Id of the DIO channel. This value will be assigned to the symbolic names.

Table 4-19. Attribute DioChannelId (DioChannel) detailed description

Property	Value
Label	Dio Channel Id
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	true
Default	0
Invalid	Range <=15 >=0

4.5.1.3 Form DioChannelGroup

A channel group represents several adjoining DIO channels represented by a logical group. This container definition does not explicitly define a symbolic name parameter, but symbolic names will be generated for each channel group. Each group provides a structure with parameters port, offset and mask.

Is included by form : [Form DioPort](#)

Figure 4-7. Tresos Plugin snapshot for DioChannelGroup form.

4.5.1.3.1 DioChannelGroupIdentification (DioChannelGroup)

A DIO channel group is identified in DIO APIs by a pointer to a data structure of type Dio_ChannelGroupType. This data structure contains the channel group information. This parameter contains the code fragment that has to be inserted in the API call of the calling module to get the address of the variable in memory which holds the channel group information, a string value should be given for this parameter. Symbolic names will be generated for each DioChannelGroup, which will be assigned with address of this string in order to point to the structure parameters. Example: OutputGroup

Table 4-20. Attribute DioChannelGroupIdentification (DioChannelGroup) detailed description

Property	Value
Label	Dio Channel Group Identification
Type	STRING
Origin	AUTOSAR_ECUC
Symbolic Name	true
Default	DioChannelGroup

4.5.1.3.2 DioPortMask (DioChannelGroup)

This shall be the mask which defines the positions of the channel group. The data type depends on the port width.

Table 4-21. Attribute DioPortMask (DioChannelGroup) detailed description

Property	Value
Label	Dio Port Mask
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	1
Invalid	Range >=1 <=65535

4.5.1.3.3 DioPortOffset (DioChannelGroup)

The position of the Channel Group on the port, counted from the LSB. This value can be derived from DioPortMask. calculationFormula = Position of the first bit of DioPortMask which is set to '1' counted from LSB

Table 4-22. Attribute DioPortOffset (DioChannelGroup) detailed description

Property	Value
Label	Dio Port Offset
Type	INTEGER
Origin	AUTOSAR_ECUC
Symbolic Name	false
Default	0
Invalid	Range >=0 <=15

How to Reach Us:**Home Page:**freescale.com**Web Support:**freescale.com/support

Information in this document is provided solely to enable system and software implementers to use Freescale products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document.

Freescale reserves the right to make changes without further notice to any products herein. Freescale makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does Freescale assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in Freescale data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. Freescale does not convey any license under its patent rights nor the rights of others. Freescale sells products pursuant to standard terms and conditions of sale, which can be found at the following address: freescale.com/SalesTermsandConditions.

Freescale, the Freescale logo, Altivec, C-5, CodeTest, CodeWarrior, ColdFire, ColdFire+, C-Ware, Energy Efficient Solutions logo, Kinetis, mobileGT, PowerQUICC, Processor Expert, QorIQ, Qorivva, StarCore, Symphony, and VortiQa are trademarks of Freescale Semiconductor, Inc., Reg. U.S. Pat. & Tm. Off. Airfast, BeeKit, BeeStack, CoreNet, Flexis, Layerscape, MagniV, MXC, Platform in a Package, QorIQ Qonverge, QUICC Engine, Ready Play, SafeAssure, SafeAssure logo, SMARTMOS, Tower, TurboLink, Vybrid, and Xtrinsic are trademarks of Freescale Semiconductor, Inc. All other product or service names are the property of their respective owners.

© 2014 Freescale Semiconductor, Inc.

Document Number UM13DIOASR4.0 Rev0003R1.0.1
Revision 1.2